Cultivo de código

Sistemas de Jardín Orientados a Objetos

Resumen: Construye un ecosistema de jardín digital completo mientras descubres conceptos avanzados de Python. Crea herramientas para gestionar jardines comunitarios de manera eficiente a través de

enfoques basados en datos.

Versión: 3.0

Contenidos

Prólogo I 2

II Instrucciones de IA 3

III Introducción 5

IV Instrucciones Generales 6

Ejercicio 0: Plantar tu Primera Semilla 7

Ejercicio 1: Organizador de Datos del Jardín 9

Ejercicio 2: Simulador de crecimiento de plantas 11

Ejercicio 3: Fábrica de plantas 13

Ejercicio 4: Sistema de Seguridad del Jardín 15

Ejercicio 5: Tipos de Plantas Especializados 17

Ejercicio 6: Analíticas del Jardín 19

XII Entrega y Envío 21

Capítulo I

Prólogo

En la era digital, incluso los jardines necesitan sistemas inteligentes.

Un jardinero sabio dijo una vez: "No puedes plantar flores y luego arrancarlas todos los días para ver cómo están las raíces." Lo mismo ocurre con el código: a veces el crecimiento más hermoso sucede cuando confiamos en el proceso y dejamos que nuestros programas se desarrollen de forma natural, capa por capa.

Detrás de cada huerto comunitario próspero se oculta una red de relaciones. Las plantas no crecen aisladas; forman comunidades donde cada especie aporta algo único mientras comparten necesidades comunes. Algunas plantas protegen a otras de plagas, algunas fijan nitrógeno en el suelo y otras proporcionan sombra a plántulas delicadas. Esta red interconectada de apoyo mutuo refleja cómo funcionan los sistemas de software bien diseñados: componentes individuales con roles distintos, trabajando juntos de manera armoniosa.

Así como un maestro jardinero sabe que un suelo saludable produce plantas sanas, los programadores experimentados entienden que un código bien estructurado crece hasta convertirse en aplicaciones robustas. Descubrirás que organizar tu código es como planificar un jardín: algunos elementos requieren protección, otros necesitan espacio para crecer, y los mejores sistemas emergen cuando todo tiene su lugar y propósito adecuados.

En este proyecto, cultivarás tus habilidades de programación mientras construyes herramientas que podrían ayudar genuinamente a que los huertos comunitarios florezcan. Cada línea de código que escribes es una semilla plantada en el suelo digital de la posibilidad.

Capítulo II

Instrucciones de IA

Contexto

Durante tu recorrido de aprendizaje, la IA puede ayudar con muchas tareas diferentes. Tómate el tiempo para explorar las diversas capacidades de las herramientas de IA y cómo pueden apoyar tu trabajo. Sin embargo, acércate a ellas con precaución y evalúa críticamente los resultados. Ya sea código, documentación, ideas o explicaciones técnicas, nunca puedes estar completamente seguro de que tu pregunta estaba bien formada o de que el contenido generado es preciso. Tus compañeros son un recurso valioso para ayudarte a evitar errores y puntos ciegos.

Mensaje principal

- Utiliza la IA para reducir tareas repetitivas o tediosas.
- Desarrolla habilidades para generar indicaciones, tanto de codificación como de no codificación que beneficiarán tu futura carrera.
- Aprende cómo funcionan los sistemas de IA para anticipar mejor y evitar riesgos comunes, sesgos y cuestiones éticas.
- Sigue desarrollando habilidades técnicas y de alto nivel trabajando con tus compañeros.
- Utiliza solo contenido generado por IA que entiendas completamente y por el que puedas asumir la responsabilidad.

Reglas para el aprendiz:

- Debes tomarte el tiempo para explorar las herramientas de IA y entender cómo funcionan, para que puedas usarlas de forma ética y reducir posibles sesgos.
- Debes reflexionar sobre tu problema antes de solicitar indicaciones; esto te ayuda a escribir indicaciones más claras, detalladas y relevantes, usando vocabulario preciso.
- Debes desarrollar el hábito de revisar, cuestionar y probar de forma sistemática todo lo generado por IA.
- Siempre busca revisión de pares no te fíes únicamente de tu propia validación.

Resultados de la fase:

- Desarrollar habilidades de prompting tanto de uso general como específicas de dominio.
- Aumenta tu productividad con el uso efectivo de herramientas de IA.
- Continúa fortaleciendo el pensamiento computacional, la resolución de problemas, la adaptabilidad y la colaboración.

Comentarios y ejemplos:

- Con frecuencia te encontrarás con situaciones exámenes, evaluaciones y más donde debes demostrar un entendimiento real. Prepárate, continúa desarrollando tanto tus habilidades técnicas como interpersonales.
- Explicar tu razonamiento y debatir con compañeros a menudo revela lagunas en tu comprensión. Haz del aprendizaje entre pares una prioridad.
- Las herramientas de IA a menudo carecen de tu contexto específico y tienden a proporcionar respuestas genéricas. Tus compañeros, que comparten tu entorno, pueden ofrecer ideas más relevantes y precisas.
- Donde la IA tiende a generar la respuesta más probable, tus compañeros pueden proporcionar perspectivas alternativas y matices valiosos. Confía en ellos como un punto de control de calidad.

✓ Buenas prácticas:

> Le pregunto a la IA: “¿Cómo pruebo una función de ordenación?” Me da algunas ideas. Las pruebo y reviso los resultados con un compañero. Refinamos el enfoque juntos.

✗ Malas prácticas:

> Pido a la IA que escriba una función completa, la copio y la pego en mi proyecto. Durante la evaluación entre pares, no puedo explicar qué hace ni por qué. Pierdo credibilidad, y fracaso en mi proyecto.

✓ Buenas prácticas:

> Uso la IA para ayudar a diseñar un analizador. Luego repaso la lógica con un compañero. Detectamos dos errores y lo reescribimos juntos: mejor, más limpio y completamente entendido.

✗ Malas prácticas:

> Dejo que Copilot genere mi código para una parte clave de mi proyecto. Compila, pero no puedo explicar cómo maneja los pipes. Durante la evaluación, no logro justificar y fracaso en mi proyecto.

Capítulo III

Introducción

¡Bienvenido a CodeCultivation!

Basándose en tus fundamentos de Python de la primera actividad, ahora afrontarás desafíos de programación más complejos creando un sistema de gestión de datos de jardín integral. Este proyecto introduce conceptos avanzados que hacen de Python una herramienta poderosa para modelar sistemas del mundo real.

Trabajarás en:

- Entender cómo se estructuran y ejecutan los programas en Python
- Organizar datos usando un enfoque orientado a objetos
- Crear componentes de código reutilizables
- Construir sistemas que puedan adaptarse y extenderse
- Proteger la integridad de los datos en entornos colaborativos
- Diseñar arquitecturas de software escalables

Cada ejercicio se construye sobre los anteriores, creando un ecosistema digital de jardín completo al final.

IMPORTANTE: este módulo comienza con la estructura básica de un programa en Python y luego progresa a la Programación Orientada a Objetos. Cada ejercicio debe contener las definiciones solicitadas y cualquier código requerido. Puedes incluir código de prueba simple al final de cada archivo usando bloques if __name__ == "__main__": para tus propias pruebas.

Capítulo IV
Instrucciones Generales

- Tu código debe estar escrito en Python 3.10+
- Tu código debe respetar los estándares del linter flake8
- Cada ejercicio debe estar en su propio archivo y directorio
- Usa convenciones de nomenclatura adecuadas: clases en PascalCase, funciones y variables en snake_case
- Todas las funciones y métodos deben incluir anotaciones de tipos: usa mypy para verificar tu código
- No necesitas manejar la validación de entradas a menos que se mencione explícitamente
- Enfócate en demostrar claramente los conceptos de programación
- Tus programas deben ejecutarse siempre sin errores

Nota del Ecosistema de Jardín Digital: Este proyecto se centra en Python conceptos de programación, desde la estructura básica del programa y progresando hacia la Programación Orientada a Objetos. Cada ejercicio introduce nuevas características mientras se construye un sistema cohesivo de gestión de jardín. Los ejercicios posteriores reutilizarán conceptos de los anteriores.

Programación Orientada a Objetos: A partir del Ejercicio 1, todos los ejercicios en este módulo requieren el uso de clases. Palabras clave de Python como ’class’ y ’def’ son palabras clave fundamentales del lenguaje y no necesitan estar listadas en las funciones autorizadas para cada ejercicio.

Para una visualización más agradable, puedes usar el método capitalize() de un objeto str (cadena). Esto no formará parte de las "Funciones autorizadas", pero aún así puedes usarlo si quieres.

Capítulo V

Ejercicio 0: Plantando tu primera semilla

	Ejercicio0
ft_garden_intro	
Directorio: ex0/	
Archivos para enviar: ft_garden_intro.py	
Autorizado: print()	

Antes de poder construir sistemas complejos de gestión de jardines, debes entender cómo funcionan los programas de Python.
Todo programa de Python necesita un punto de inicio, un lugar donde comienza la ejecución. ¡Es como plantar tu primera semilla en un jardín!

Tu tarea es crear tu primer programa en Python que muestre información sobre una planta en tu jardín.

Requisitos:

- Crea un programa que se ejecute directamente al ser ejecutado con python3 ft_garden_intro.py
- Usa el patrón especial if __name__ == "__main__":.
- Almacena la siguiente información de la planta en variables simples: nombre, altura y edad.
- Muestra la información de la planta usando print()
- Imita la salida de la pantalla a continuación; no habrá una verificación estricta de la salida durante la evaluación.

Ejemplo:

```
$> python3 ft_garden_intro.py
=== Bienvenido a Mi Jardín
=== Planta: Rosa Altura: 25cm
Edad: 30 días


=== Fin del programa ===
```

Comprender cómo comienzan y se ejecutan los programas es fundamental antes de pasar a conceptos más complejos como clases y otras características orientadas a objetos.

Nota: A partir de este ejercicio y para todos los siguientes ejercicios en este módulo (y proyectos futuros de Python), se le permite usar if __name__ == "__main__": bloques para probar su código. Este patrón no se mostrará explícitamente en las funciones autorizadas, pero permanece disponible durante todo el proyecto.

Debe ser capaz de explicar por qué es importante utilizar la línea if __name__ == "__main__":. Esto lo llevará al concepto de importaciones, que formará parte de un próximo proyecto. Sin embargo, es importante introducir esta buena práctica ahora.

¿Has notado que algunos archivos de Python empiezan con una línea especial que comienza con #!? Investiga qué hace esta línea “shebang” y por qué podría ser útil para que tus scripts sean ejecutables directamente. La evaluación te pedirá actualizar tu script en vivo para usar este “shebang”.

Capítulo VI

Ejercicio 1: Organizador de Datos del Jardín

	Ejercicio1
ft_garden_data	
Directorio: ex1/	
Archivos a enviar: ft_garden_data.py	
Autorizado: print()	

El jardín comunitario necesita rastrear varias plantas con su información. Debes almacenar y mostrar datos de varias plantas de forma eficiente.

Cada planta tiene:

- Un nombre
- Altura en centímetros
- Edad en días

Crea un programa que gestione datos de al menos 3 plantas diferentes y muestre su información de manera organizada.

Requerido: Debes crear una clase Plant que sirva como modelo para cualquier planta, en lugar de manejar cada una individualmente. Todas las plantas se describirán usando las mismas en-atributos o características, listados arriba: nombre, altura y edad. Para cada planta, la clase se instanciará y luego los atributos se establecerán en sus valores específicos.

Crearás un método show() en la clase para mostrar la información de la planta.

Ejemplo:

```
$> python3 ft_garden_data.py
=== Registro de Plantas del
Jardín === Rosa: 25cm, 30
días, Cactus: 15cm, 120 días
```

¿Cómo estás almacenando actualmente la información de las plantas? Hay varias formas de hacerlo. ¿Qué desafíos podrían surgir con más plantas? En los próximos ejercicios, es posible que necesites hacer evolucionar tu enfoque.

Si ya has trabajado con programación en Python o con programación orientada a objetos en el pasado, probablemente irás directamente a una solución avanzada. En este caso, puede que encuentres los siguientes ejercicios un poco redundantes.

Capítulo VII

Ejercicio 2: Simulador de Crecimiento de Plantas

	Ejercicio2
ft_crecimiento_planta	
Directorio: ex2/	
Archivos para enviar: ft_crecimiento_planta.py	
Autorizado: print(), range(), round()	

El administrador del jardín quiere simular el crecimiento de las plantas a lo largo del tiempo. Debes rastrear cómo cambian las plantas y proporcionar operaciones para modificar su estado.

Requisitos:

- Reutiliza tu clase Plant del Exercise 1
- Las plantas deben poder crecer() y envejecer() por sí mismas (es decir, serán métodos de la clase)
- Simula una semana de crecimiento para una planta, luego accede a los datos de la clase para obtener la altura final y mostrar el aumento total de la semana.
- Considera cómo debe cambiar la planta con el tiempo al usar grow() y age(). Diferentes plantas pueden evolucionar de manera diferente.

Tu programa debe mostrar una planta cambiando con el tiempo y un resumen al final de la semana. Piensa en dar a tus plantas comportamientos que hagan crecer() de manera diferente.

Ejemplo:

```
$> python3 ft_plant_growth.py
=== Crecimiento de la Planta
===
```

Existen múltiples enfoques posibles para manejar esto. Puedes implementarlo de la manera que desees, añadiendo atributos a la clase Plant, parámetros a los métodos de la clase, etc.

Capítulo VIII

Ejercicio 3: Fábrica de Plantas

	Ejercicio3
ft_fábrica_de_plantas	
Directorio: ex3/	
Archivos a enviar: ft_plant_factory.py	
Autorizado: print(), range(), round()	

El centro de jardinería necesita crear muchas plantas rápidamente utilizando su clase Plant con la dif-
valores iniciales diferentes. Debe optimizar el proceso de creación de instanciar y
plantas: inicializar la clase para una nueva planta al mismo tiempo.

Requisitos:

- Las plantas deben crearse directamente con su información inicial (nombre, altura inicial, edad inicial)
- Cada planta debe estar lista para usar inmediatamente después de la construcción (p. ej., si quieres hacerla crecer())
- Crea al menos 5 plantas diferentes con características variadas
- Muestra todas las plantas creadas en un formato organizado

Mejora tu clase Plant de los ejercicios anteriores. Ten en cuenta que ya puede que hayas optimizado el proceso de creación de plantas en el Ejercicio 1 si ya estabas familiarizado con la programación orientada a objetos.

Ejemplo:

```
$> python3 ft_plant_factory.py === Salida de la
Factoría de Plantas === Creado: Rosa: 25.0cm,
30 días de edad Creado: Roble: 200.0cm, 365
días de edad Creado: Cactus: 5.0cm, 90 días de
edad Creado: Girasol: 80.0cm, 45 días de edad
Creado: Helecho: 15.0cm, 120 días de edad
```

Deberías poder usar el método show() sin cambios para mostrar la salida anterior.

Capítulo IX

Ejercicio 4: Sistema de Seguridad del Jardín

	Ejercicio4
ft_garden_security	
Directorio: ex4/	
Archivos para enviar: ft_garden_security.py	
Autorizado: print(), range(), round()	

El gerente del jardín está preocupado por la integridad de los datos. Algunos voluntarios, sin querer, corrompieron los datos de las plantas al establecer valores inválidos (alturas negativas, edades imposibles). Necesitas crear un sistema seguro que proteja y encapsule los datos sensibles.

Requisitos:

- Mejorar tu clase Plant para proteger sus datos de la corrupción
- Proporcionar formas controladas de modificar los datos de la planta: set_height(), set_age()
- Proporcionar formas seguras de acceder a los datos de la planta: get_height(), get_age()
- Asegurar que la altura de la planta no pueda ser negativa mediante validación
- Asegurar que la edad de la planta no pueda ser negativa mediante validación
- Imprimir mensajes de error desde la clase cuando se proporcionen valores inválidos, dejando los datos sin cambios o creando la planta con valores predeterminados
- Usar encapsulación: evitar que tus atributos de clase se utilicen directamente usando la convención de “protegido” (no el mangling)

Considera cómo podrías crear un sistema que valide los datos antes de almacenarlos, asegurando la integridad de los datos.

Ejemplo:

```
$> python3 ft_garden_security.py === Sistema de
Seguridad del Jardín === Planta creada: Rosa:
15.0cm, 10 días

Altura actualizada: 25cm
Edad actualizada: 30 días

Rosa: Error, la altura no puede ser negativa.
Actualización de altura rechazada. Rosa: Error, la
edad no puede ser negativa. Actualización de
edad rechazada

Estado actual: Rosa: 25.0cm, 30 días
```

Capítulo X

Ejercicio 5: Tipos de Plantas Especializados

	Ejercicio5
ft_plant_types	
Directorio: ex5/	
Archivos a enviar: ft_plant_types.py	
Autorizado: super(), print(), range(), round()	

El jardín ahora necesita manejar diferentes tipos de plantas: flores, árboles y verduras. Cada tipo tiene características únicas, pero hereda características de planta comunes de su categoría padre.

Requisitos:

- Comienza con tu clase Plant del ejercicio anterior, la cual contiene las características comunes (nombre, altura y edad)
- Crea tipos especializados: Flor, Árbol y Vegetal
- Cada tipo especializado debe heredar las características básicas de planta
- La Flor necesita: un atributo de color y la capacidad de florecer()
- El Árbol necesita: un atributo de diámetro de tronco y la capacidad de producir_sombra()
- El Vegetal necesita: un atributo de temporada de cosecha y un valor nutricional
- Al crear plantas especializadas, llama a los métodos de la clase padre desde tu nueva clase usando super(). Puede aplicarse a cualquier método, incluido __init__()
- Una llamada a show() en una clase especializada debe imprimir la salida estándar de Plant y las características adicionales de tu planta especializada. Tu anulación de método puede reutilizar el código ya existente en la clase padre.
- Crea al menos una instancia de cada tipo de planta; haz que la flor florezca; haz que el valor nutricional empiece en 0, y luego aumente cuando se llamen a los métodos age() y grow() de la verdura.

- Evitar duplicar código común de plantas entre diferentes tipos especializados.
- No es necesario validar los nuevos atributos en las tres clases nuevas.

Ejemplo:

```
  $> python3 ft_plant_types.py
=== Tipos de Plantas de Jardín ===
=== Flor
Rosa: 15.0cm, 10 días de edad
 Color: rojo
 La rosa aún no ha florecido
 pidiéndole a la rosa que florezca]
Rosa: 15.0cm, 10 días de edad
 Color: rojo
 ¡La rosa está floreciendo hermosamente!

=== Árbol
Roble: 200.0cm, 365 días de edad
 Diámetro del tronco: 5.0cm
 pidiendo al roble que produzca sombra ]
El roble ahora produce una sombra de 200.0cm de largo y 5.0cm de ancho.

=== Verdura
Tomate: 5.0cm, 10 días de edad
 Temporada de cosecha: abril
 Valor nutricional: 0
hacer que el tomate crezca y envejezca durante 20 días]
Tomate: 47.0cm, 30 días de edad
 Temporada de cosecha: abril
 Valor nutricional: 20
```

¿Cómo estás manejando las características comunes compartidas por todos los tipos de plantas? Se recomienda evitar repetir el mismo código en las diferentes clases para manejar estas características comunes.

Capítulo XI

Ejercicio 6: Analítica de Jardín

	Ejercicio6
ft_analítica_jardín	
Directorio: ex6/	
Archivos a entregar: ft_garden_analytics.py	
Autorizado: super(), print(), range(), round(), staticmethod(), classmethod()	

Mejora tus clases del ejercicio anterior para mostrar analíticas. Necesitarás manejar relaciones de datos complejas, usar componentes anidados y cadenas de herencia.

Requisitos:

- Crea un método estático para la clase Plant que verifique si una edad específica dada como parámetro es mayor de un año.
- Crea un método de claseque te permita crear una planta anónima directamente cuando aún no tienes toda la información.
- Crea una clase Seed que herede de Flower y que almacene el número de semillas una vez que la flor ha florecido. El método show() debe mejorarse en consecuencia.
- Cada Planta tiene un sistema interno, implementado como un anidado clase, que almacena datos estadísticos: número de llamadas a grow(), número de llamadas a age(), número de llamadas a show(). Se requiere encapsulamiento, así como una función de visualización.
- Los árboles necesitan una pieza extra de datos estadísticos: número de llamadas a produce_shade().
- Finalmente, crea una función única, que no forme parte de ninguna clase, que muestre estadísticas para cualquier tipo de planta.

Nota: la sintaxis de decoradores también es aceptada para métodos estáticos y de clase

Ejemplo:

```
$> python3 ft_garden_analytics.py
=== Estadísticas del jardín ===
=== Verificación de años ===
¿30 días más que un año? -> Falso
¿400 días más que un año? -> Verdadero

=== Flor
Rosa: 15.0 cm, 10 días de edad
  Color: rojo
  La rosa aún no ha florecido
[estadísticas de Rosa]
Estadísticas: 0 crecimiento, 0 edad, 1 exhibición
[pidiendo a la rosa que crezca y florezca]
Rosa: 23.0 cm, 10 días de edad
  Color: rojo
  ¡La rosa está floreciendo hermosamente!
[estadísticas de Rosa]
Estadísticas: 1 crecimiento, 0 edad, 2 exhibiciones

=== Árbol
Roble: 200.0 cm, 365 días de edad
  Diámetro del tronco: 5.0 cm
[estadísticas de Roble]
Estadísticas: 0 crecimiento, 0 edad, 1 exhibición
  0 sombra
[pidiendo al roble que produzca sombra]
El roble ahora produce una sombra de 200.0 cm de largo y 5.0 cm de ancho.
[estadísticas de Roble]
Estadísticas: 0 crecimiento, 0 edad, 1 exhibición
  1 sombra

=== Semilla
Girasol: 80.0 cm, 45 días de edad
  Color: amarillo
  El girasol aún no ha florecido
  Semillas: 0
[hacer que el girasol crezca, envejezca y florezca]
Girasol: 110.0 cm, 65 días de edad
  Color: amarillo
  ¡El girasol está floreciendo hermosamente!
  Semillas: 42
[estadísticas del Girasol]
Estadísticas: 1 crecimiento, 1 edad, 2 exhibiciones

=== Anónimo
Planta desconocida: 0.0 cm, 0 días de edad
[estadísticas de planta desconocida]
Estadísticas: 0 crecimiento, 0 edad, 1 exhibición
```

Este ejercicio reúne todos los patrones de programación que has aprendido a lo largo del proyecto. Serás evaluado en tu comprensión de cómo interactúan y se organizan los distintos componentes dentro de un sistema complejo.

Capítulo XII

Entrega y Presentación

Entregue su tarea en su repositorio de Git como siempre. Solo el trabajo dentro de su repositorio será evaluado durante la defensa. No dude en verificar dos veces los nombres de sus archivos para asegurarse de que sean correctos. También verifique los estándares de Python y las anotaciones de tipos usando flake8 y mypy.

Durante la evaluación, es posible que se le pida explicar conceptos de programación, demostrar relaciones de herencia o ampliar su código con nueva funcionalidad. Asegúrese de entender los principios detrás de sus implementaciones.

Solo necesita enviar los archivos solicitados por la asignatura de este proyecto. Enfoque en código limpio y bien documentado que demuestre claramente los principios de la programación.